

An AI-Augmented Real-time Quiz Platform for Monitoring and Analyzing Students' Understanding in Live Classrooms

Nithis Manora¹ and Hideyuki Takajo^{1*}

¹Department of Communication Network Engineering, Kagawa College, National Institute of Technology, 551 Kohda, Takuma, Mitoyo, Kagawa, Japan

*Corresponding author's e-mail: takajo@cn.kagawa-nct.ac.jp

Grasping students' understanding level during a class — rather than after it — remains a central challenge in classroom teaching. Building on an earlier real-time monitoring system, we developed an AI-augmented, web-based quiz platform that lets teachers observe every student's answer second-by-second, detect hidden confusion, and obtain automated pedagogical analysis without sending any student data outside the classroom. The platform combines four communication channels — REST for persistent data, WebSocket (Socket.IO) for live answers, Server-Sent Events for streamed analysis, and direct calls from the web tier to a local large language model (LLM) served by Ollama. A document database (MongoDB) stores quizzes, archived results, and event-level interaction logs, while an in-memory store (Redis) holds live session state with an automatic in-memory fallback for fault tolerance. Beyond binary correctness, a Confusion Detection Engine infers genuine understanding from answer-change behaviour and response time, and a three-level AI analysis pipeline (per-student, per-class, and cross-session) produces multilingual reports through a queue-stream-cache mechanism that supports an entire class requesting analysis at once. We describe the architecture, data flow, and core algorithms in detail, report a classroom demonstration with Japanese students, and extend the previously proposed *Test-Driven Education* method into an AI-assisted formative-feedback loop.

Keywords: *real-time monitoring; formative assessment; confusion detection; local large language model; WebSocket; learning analytics*

I. INTRODUCTION

Tablets and personal devices are now common in places of education, and a large body of work has examined how mobile and computer-based tools affect learning. Existing e-learning systems already provide quiz tests, content management, and grade management, and students can use them both in class and at home. In practice, however, teachers seldom consult quiz results during the lesson itself: with paper tests, marking is slow, so explanations of misunderstood points are postponed to the next class; with conventional online tests, answers are typically transmitted only after the student presses a final “submit” button, so the teacher cannot see how understanding evolves question by question.

An earlier prototype from our laboratory addressed part of this gap by monitoring students' answers in real time over a two-way (WebSocket) channel, allowing a teacher to re-explain difficult points within the same class. That system established the value of immediacy, but it was limited to four-choice questions, reported only correct/incorrect status, and offered no automated interpretation of the data it collected. Two developments since then motivate the present work. First, classroom analytics has moved from simply scoring answers toward understanding the learning process — a great deal of research now estimates students' affective and cognitive states from eye movement, facial expression, physiological signals, or mouse and keystroke dynamics. Many of these signals, however, require additional sensors or cameras and raise privacy concerns. Second, large language models (LLMs) have made it feasible to turn raw interaction data into readable pedagogical advice, but cloud-

hosted models conflict with the privacy expectations of a classroom that handles minors' data.

In this paper we present a real-time quiz platform, hereafter referred to as the *system*, that advances the earlier prototype in three directions. (1) It infers *genuine understanding* rather than mere correctness, through a Confusion Detection Engine driven by behavioural signals (answer changes and timing) that need no extra hardware. (2) It performs automated pedagogical analysis at three levels — individual student, whole class, and across repeated sessions — using a *local* LLM, so that no student data leaves the laboratory network. (3) It is engineered for real classroom use at scale, with a fault-tolerant cache, a waiting-room and gamified projector view to organise the class, and a queue-stream-cache pipeline that lets a whole class request AI feedback simultaneously without overloading the model. We describe the system architecture and the data flow in detail, and we extend our previously proposed teaching method, *Test-Driven Education*, into an AI-assisted feedback loop.

II. MATERIALS AND METHOD

A. System Overview and Architecture

As shown in Figure 1, the system is a web application composed of a front-end tier, a back-end tier, two data stores, and a local AI engine. The front end is built with Next.js and React and is served to ordinary web browsers, so teachers and students need no native application — a tablet, laptop, or smartphone browser is sufficient. The back end is a Node.js server using the Express framework together with Socket.IO. As in the earlier prototype, the server runtime is

single-threaded with non-blocking I/O, which gives quick processing and response under many concurrent connections. Two NoSQL data stores are used for complementary purposes: **MongoDB**, a document-oriented database that stores hierarchically structured data in JSON form (quizzes, archived results, and interaction logs), and **Redis**, an in-memory key-value store used for live session state that must be read and written the instant a student answers.

A distinctive feature of the system is that it uses four cooperating communication channels rather than a single request–response protocol, each matched to a different need:

- REST over HTTP/JSON for create–read–update–delete operations on quizzes, authentication (JSON Web Tokens), dashboards, and session history;

- WebSocket (Socket.IO) for the live examination room, where each answer is pushed to the server and broadcast to the teacher’s screen in real time;
- Server-Sent Events (SSE) for streaming AI-generated text from the back end token by token, so a teacher sees an analysis appear progressively rather than after a long wait; and
- direct calls from the Next.js server tier to the local LLM for the in-app assistant and per-student analysis, streamed back to the browser as newline-delimited JSON (NDJSON).

The technologies adopted in each layer are summarised in Table 1. All of them are open-source and run on commodity hardware inside the classroom or laboratory.

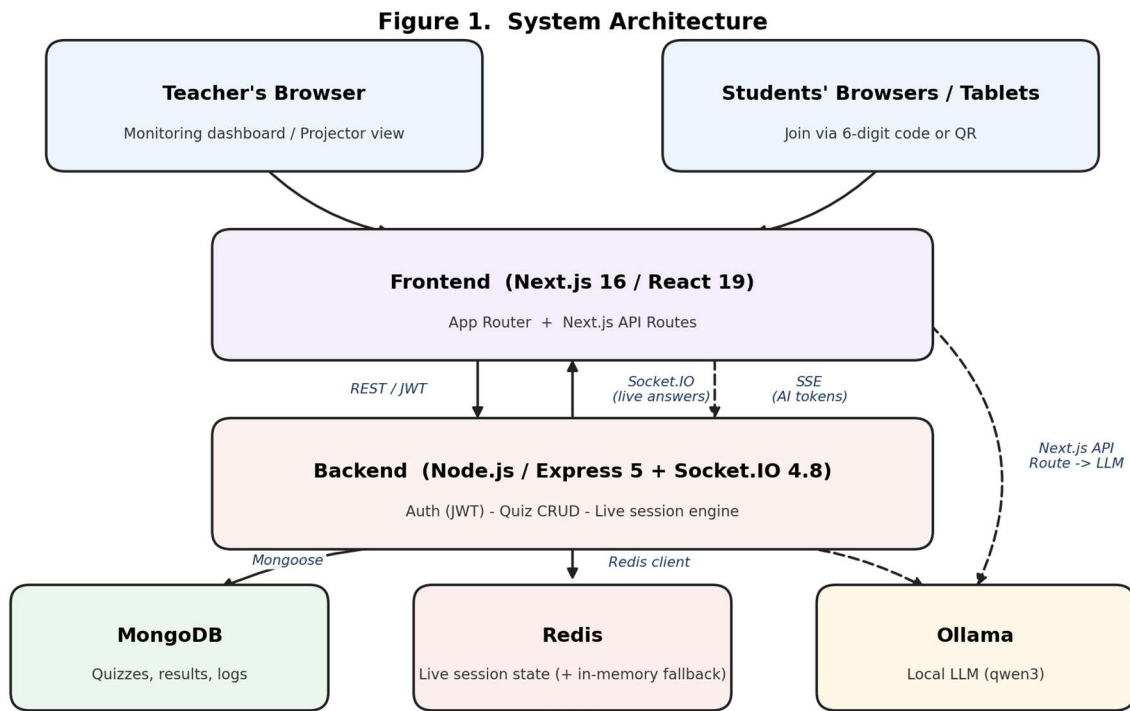


Figure 1. System architecture. Four communication channels connect the browser clients, the Next.js front end, the Express/Socket.IO back end, and the data and AI tiers (MongoDB, Redis, and a local Ollama LLM).

Table 1. Technologies adopted in each layer of the system.

Layer	Technology
Front end	Next.js 16 (App Router), React 19, TypeScript, TailwindCSS, Zustand, Recharts, socket.io-client
Back end	Node.js, Express 5, Socket.IO 4.8, JWT + bcrypt, Mongoose
Database	MongoDB — quizzes, archived session results, interaction logs
Cache	Redis — live session state with automatic in-memory fallback
AI engine	Ollama (local LLM, e.g. qwen3) streamed via SSE / NDJSON

B. Quiz Authoring and the AI Quiz Copilot

Before any examination, a teacher authors quizzes in a dedicated **Quiz Builder** (Figure 2). Each quiz carries a title,

description, subject, chapter, difficulty (easy / medium / hard), an optional time limit and duration, free-form tags, and a switch controlling whether answers are revealed to students after the quiz. Questions are either multiple-choice

or true / false; the teacher can attach an image to a question or to an individual choice, mark the correct choice, and reorder questions by drag-and-drop. A live preview and continuous client-side state management keep the form responsive while a large quiz is being assembled.

To accelerate authoring, the builder embeds an **AI Quiz Copilot** — a chat assistant backed by the same local LLM (here, qwen3). The teacher types a natural-language instruction such as “create five questions about web application basics,” and the assistant replies with a structured block that the front end parses into a summary card; a single click pours the entire set — questions, choices, correct answers, subject, and chapter — into the form (Figure 2). Because the model runs locally, quiz drafts are produced without sending the prompt to any external service. The teacher then reviews and edits the draft, adjusts the answer key where needed, and publishes the quiz, which generates the six-character access code and QR code used by students to join.

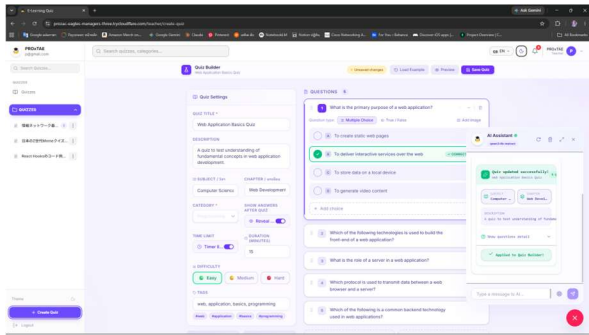


Figure 2. Quiz Builder with the AI Quiz Copilot. A natural-language request is turned by the local LLM into a structured draft that is applied to the form in one click; the teacher then reviews, edits the answer key, and publishes.

C. Persistent and Live Data: Division of Storage

The system deliberately separates durable data from volatile live state. Quizzes authored by teachers, archived results of finished examinations, and the event-level interaction logs are stored permanently in MongoDB. The state of an examination that is currently in progress — who is online, what each student has answered to each question, how many times they changed their mind, and the running statistics — is kept in Redis. The reason mirrors the rationale of the earlier prototype: an in-memory store can be accessed fast enough to record an answer the moment a student selects a choice, so the teacher can monitor the class second-by-second. Each live key carries a 24-hour time-to-live that is refreshed on activity; when the teacher ends the session, the live state is archived to MongoDB and the Redis keys are cleared.

Storing every answer immediately also provides resilience. If the teacher’s screen momentarily loses the connection or the page is refreshed, the full dashboard can be reconstructed from Redis, so monitoring continues without data loss. Should Redis itself become unavailable, the server transparently falls back to an equivalent in-

memory map and the examination proceeds; Redis is warmed up again in the background. Crucially, the server never trusts a correctness flag sent by the client: every submitted answer is re-validated against the authoritative answer key in the database before it is recorded.

D. Real-time Live Monitoring and the Answer Popover

When a quiz is published, the system generates a six-character access code (ambiguous characters such as I, O, 0 and 1 are removed) and a QR code; students open the link, enter a name, and join without creating an account. On the student’s device each question is answered by a single tap, with no submit button — the answer is sent the instant a choice is made (Figure 4) — so changes of mind are captured as they happen rather than only at the end. Each answer travels over WebSocket to the server, which validates it against the key, stores it in Redis, recomputes class statistics, and broadcasts the update to the teacher’s room. The teacher’s console centres on a **Matrix Grid** in which each row is a student and each column is a question; a cell encodes the answer state (correct, incorrect, in progress, or not yet reached). Alongside the grid, a live leaderboard, a statistics panel (average score, progress, correct ratio, efficiency), and answer-timeline charts update continuously, and the whole dashboard is recoverable from Redis so the teacher may refresh the page at any time without losing the view.

Clicking any cell opens an **Answer Popover (Student Analytics)** for that student and question. Rather than a single correct/incorrect mark, the popover reports the question text and the student’s selected choice against the key, the *response time*, the number of *answer changes*, an accuracy verdict, and a high / low *confusion* badge, together with an *answer-evolution timeline* that plots every change the student made over the course of the question. A single cell therefore becomes a detailed record of how an answer was reached — for example, a correct answer that was changed six times over twenty-seven seconds is flagged as high-confusion — which is precisely the kind of process information that conventional “submit at the end” tests discard.

E. Confusion Detection Engine

A central limitation of correct/incorrect reporting is that a correct answer does not necessarily indicate understanding. The system therefore estimates a confusion level for every answer from two behavioural signals that require no additional hardware: the number of times the student changed the answer, and the time taken to answer. The rule is intentionally simple and transparent so that teachers can interpret it:

```
if changes ≥ 2 or time > 60 s → high; else if
changes ≥ 1 or time > 30 s → low; otherwise →
none.
```

The resulting level is shown as a badge in the grid, accumulated per question as a confusion count in the archive, and recorded for each answer in the interaction log,

where it later becomes an input to the AI analysis. This makes it possible to identify a student who reaches the correct answer only after repeated changes — a likely sign of guessing or fragile understanding — and to single out questions that confuse much of the class even when most students eventually answer them correctly.

F. Event-level Interaction Logging

While a student works, the client records every event for each question — viewing the question, the first selection, each change (keeping both the previous and new choice), de-selection, and periodic on-task / off-task heartbeats — each with a timestamp and the time spent on the question. On submission, the events are assembled into a single JSON document (session metadata, per-question answer logs, and a summary) and sent to the server, which again re-validates every answer against the official key before storing it. These logs are the raw material for individual AI analysis: for example, a long time on one question accompanied by repeated off-task heartbeats indicates distraction rather than a hard question, a distinction the teacher could not otherwise make.

G. Three-level AI Analysis with a Local LLM

The system turns the data it collects into pedagogical advice at three levels, all produced by a *local* LLM served by Ollama so that student data never leaves the local network. (1) **Per-student analysis** builds a structured prompt from a student's interaction log — score, timing, confusion rate, and per-question flags such as “answered unusually fast” or “hesitated” — and returns a five-part report (overall assessment, strengths, weak areas, study recommendations, and encouragement). (2) **Per-class analysis** summarises a whole session: an overview, the three hardest questions with reasons, the three best-answered questions, concrete teaching recommendations, and a strategy for the next lesson. (3) **Cross-session analysis** compares repeated runs of the same quiz across different classes to reveal whether a concept is “universally difficult” or specific to one group. Every report can be generated in Thai, English, or Japanese, the chosen language being forced through to the model's prompt.

A practical difficulty is that an entire class may request analysis at the same moment, which a single local model cannot serve concurrently. The system addresses this with three combined mechanisms. A cache stores each generated report (per log and per language) in MongoDB, so a repeat or refresh returns the previous result instantly without touching the model. A semaphore queue limits how many requests reach the model at once and streams a live queue position to those waiting. Streaming itself delivers the answer token by token as NDJSON, so the first users see text appear immediately while the rest wait in an orderly queue; when a report finishes, it is written to the cache. Together these let a class of twenty-five students request feedback at once without the model failing.

H. Session Lifecycle: Waiting Room, Projector View, and Clean Rounds

Before a quiz begins, students enter a **waiting room** that shows the live list of participants with their avatars and a “ready” toggle, so the teacher can wait until the class is assembled before starting everyone simultaneously. A separate full-screen **projector view** can be cast to a classroom screen, presenting the same information — and, during the quiz, a gamified progress roadmap and a live leaderboard — to motivate the class. Each examination is identified by its own session token; when the teacher ends a session, the system archives the results and rotates the token, so the next round starts cleanly without stale names or answers carried over. The teacher may also remove a duplicate or stuck student from the session, keeping the grid and statistics accurate.

III. RESULT AND DISCUSSION

We evaluated the system in a demonstration with Japanese students using quizzes on basic information and networking. Quizzes were prepared in advance in the Quiz Builder; the teacher published one, students joined from their own browsers using the access code or QR code (Figure 3 shows the waiting room), and the examination was started for everyone at once. On the student side, each question was answered by a single tap with no submit button (Figure 4), so every selection — and every change of mind — reached the server immediately. Throughout the session the teacher's Matrix Grid (Figure 5) updated the instant each student answered, with cell colours changing immediately between correct, incorrect, and in-progress states. As with the earlier prototype, this immediacy let the teacher identify which questions the class struggled with and re-explain them during the same lesson — but the present system added several capabilities the prototype lacked.

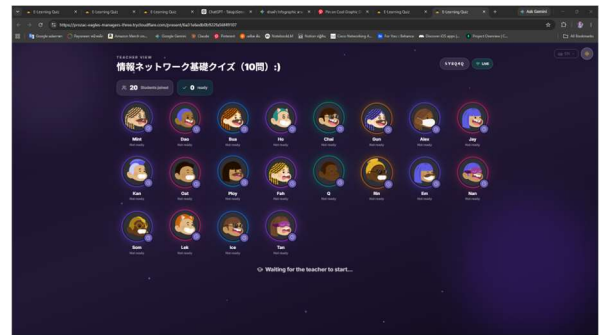


Figure 3. Waiting room. Students appear live with avatars as they join via the access code or QR code; the teacher starts the quiz for everyone at once.

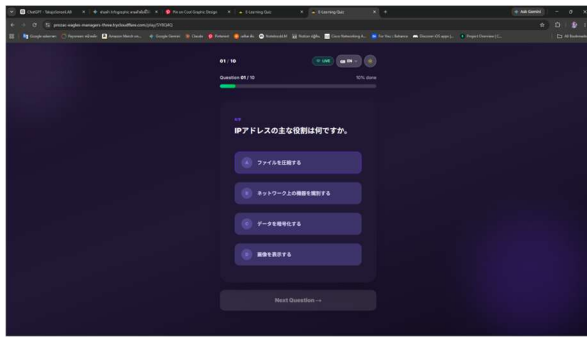


Figure 4. Student's answering view. Each question is answered by a single tap with no submit button, so the choice — and any later change — is sent to the server the moment it is made.

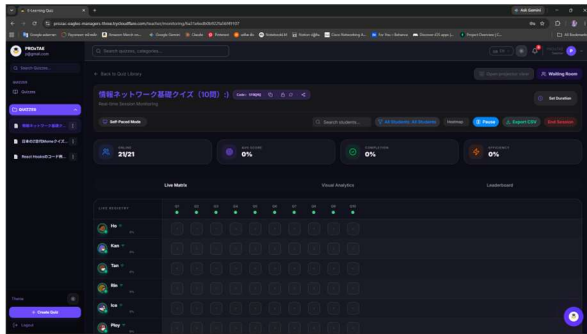


Figure 5. Teacher's live monitoring console. The Matrix Grid (students $\times$ questions) updates the moment each answer arrives, alongside live statistics and a leaderboard; clicking a cell opens the Answer Popover with response time, answer changes, and confusion.

First, the Confusion Detection Engine surfaced students whose correct answers concealed weak understanding. Several students reached a correct answer only after two or more changes, raising a high-confusion badge in the Answer Popover (for instance, a correct answer changed six times over twenty-seven seconds); without this signal the teacher would have judged them to have mastered the question. Second, after the quiz, students and teacher could request AI analysis. Figure 6 shows a per-student report generated locally from a student's interaction log, and Figure 7 shows a class-level summary streamed token by token. Because the model runs locally, these reports were produced without sending any answer data outside the room.

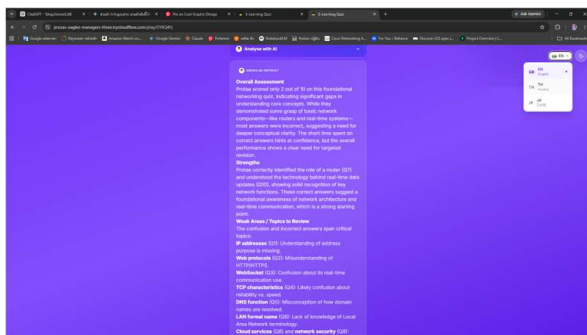


Figure 6. Personalised AI analysis for one student, generated locally from the interaction log: overall assessment, strengths, weak areas, study recommendations, and encouragement.

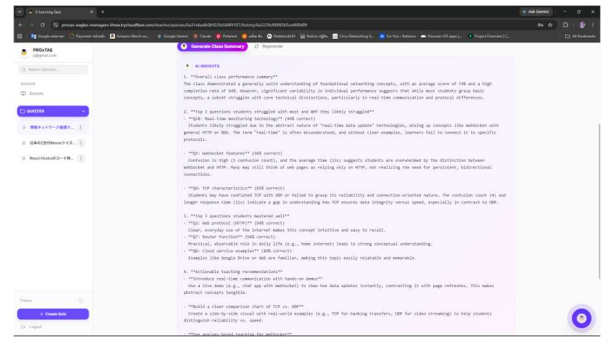


Figure 7. Class-level AI summary, streamed token by token over SSE: hardest questions, best-answered questions, and concrete teaching recommendations.

Third, the waiting room and the gamified projector view (Figure 8) helped organise the class and sustain engagement, addressing the well-known observation that novelty alone does not keep students interested. Finally, when the same quiz was run for more than one group, the cross-session comparison (Figure 9) and its AI trend analysis distinguished concepts that were difficult for every group from those specific to one class. The student's own result screen (Figure 10) closed the loop by showing each learner their score and per-question review immediately.

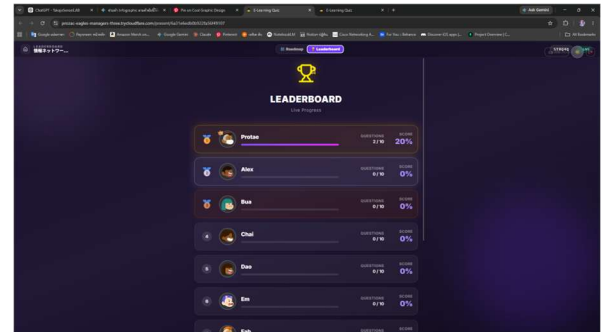


Figure 8. Gamified projector view: a live leaderboard and progress roadmap cast to the classroom screen to organise and motivate the class.

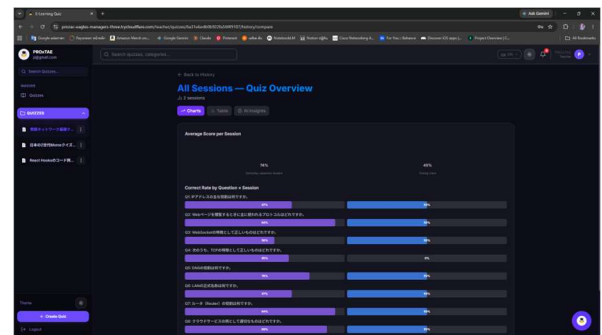


Figure 9. Cross-session comparison: per-question correct rates across repeated runs of the same quiz, with AI trend analysis identifying universally difficult concepts.

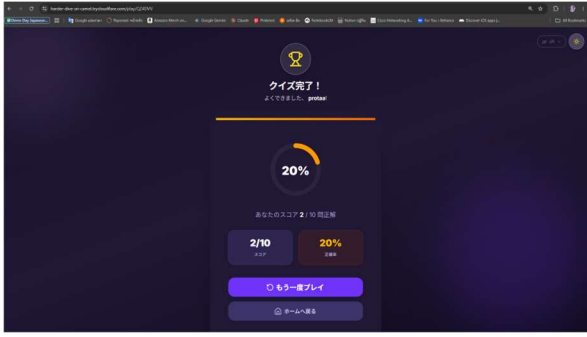


Figure 10. Student result screen shown immediately after the quiz, with score and per-question review.

Two engineering observations are worth noting. The automatic in-memory fallback meant that a transient cache problem did not interrupt an examination, and server-side re-validation of every answer guarded the statistics against manipulated clients. The queue-stream-cache pipeline allowed a whole class to request AI feedback at once: the first requests began streaming immediately, later ones displayed a decreasing queue position, and any repeated request returned instantly from cache. These properties matter for real classroom use, where reliability under simultaneous load is as important as the analysis itself.

IV. CONCLUSION

We have developed and demonstrated an AI-augmented, web-based quiz platform that monitors students' understanding in real time during a class. Beyond reporting correctness, the system infers genuine understanding through a transparent confusion-detection rule, logs interaction at the event level, and produces multilingual pedagogical analysis at the level of the individual student, the whole class, and across repeated sessions — all using a local LLM, so that student data never leaves the classroom network. A fault-tolerant cache, server-side re-validation, a waiting room and gamified projector view, and a queue-stream-cache pipeline make the platform usable for an entire class at once. In the demonstration we could see which parts of the lesson the students understood and which they did not, identify confusion hidden behind correct answers, and obtain teaching recommendations during the same class — outcomes that were difficult with conventional e-learning tools.

The platform also extends a teaching method we previously proposed, Test-Driven Education, which draws an analogy between software development and classroom teaching (Table 2). In test-driven development, a developer first writes a test, then writes code to pass it, and improves the code by repetition; by analogy, a teacher prepares short quizzes — used for monitoring, not grading — and, by repeating brief quiz-and-explain cycles, gradually teaches more advanced content according to the class's understanding. The present system strengthens this loop in two ways: the confusion signal tells the teacher not only what was answered but how confidently, and the local AI converts each cycle's data into immediate, actionable feedback. As future work we will study the effectiveness of this AI-assisted Test-Driven Education in regular classes, broaden the question types beyond multiple choice, and evaluate the platform in remote and hybrid lectures, for which immediate two-way feedback is especially valuable.

Table 2. Analogy between the software-development process and a class, extended with the test-driven feedback loop.

Software development	Traditional class	Test-driven loop
Define requirements	Determine class contents	Prepare short quizzes
Design	Plan a class	Plan a quiz-and-explain cycle
Programming	Give a class	Teach, then re-explain
Test	Examination (quiz)	Monitor understanding, repeat

V. ACKNOWLEDGEMENT

This study was carried out in the Department of Communication Network Engineering, Kagawa College, National Institute of Technology. The system reported here has been developed using many open-source projects, including Node.js, Next.js, React, Express, Socket.IO, MongoDB, Redis, and Ollama. We thank their authors and communities.

VI. REFERENCES

- [1] Algoufi, R 2016, 'Using Tablet on Education', World Journal of Education, vol. 6, no. 3, pp. 113-119.
- [2] Beck, K 2003, Test-driven development by example, Addison-Wesley, Boston.
- [3] Bell, TE & Thayer, TA 1976, 'Software requirements: Are they really a problem?', Proceedings of the 2nd International Conference on Software Engineering (ICSE '76), pp. 61-68.
- [4] Brown, E 2014, Web Development with Node and Express, O'Reilly Media, Sebastopol, CA.
- [5] Chua, YP 2012, 'Effects of computer-based testing on test performance and testing motivation', Computers in Human Behavior, vol. 28, no. 5, pp. 1580-1586.
- [6] Crompton, H & Burke, D 2018, 'The use of mobile learning in higher education: A systematic review', Computers & Education, vol. 123, pp. 53-64.
- [7] Deshmukh, SP, Patwardhan, MS & Mahajan, AR 2018, 'Feedback based real time facial and head gesture recognition for e-learning', Proceedings of the ACM India Joint International Conference on Data Science and Management of Data (CoDS-COMAD '18), pp. 360-363.
- [8] Fette, I & Melnikov, A 2011, The WebSocket Protocol, RFC 6455, viewed 6 July 2019, <<https://tools.ietf.org/html/rfc6455>>.

- [9] Khanam, Z & Ahsan, MN 2017, 'Evaluating the Effectiveness of Test Driven Development: Advantages and Pitfalls', *International Journal of Applied Engineering Research*, vol. 12, no. 18, pp. 7705-7716.
- [10] Major, L, Haßler, B & Hennessy, S 2017, 'Tablet Use in Schools: Impact, Affordances and Considerations', in *Handbook on Digital Learning for K-12 Schools*, Springer, Cham, pp. 115-128.
- [11] Muldner, K, Christopherson, R, Atkinson, R & Burleson, W 2009, 'Investigating the Utility of Eye-Tracking Information on Affect and Reasoning for User Modeling', in *User Modeling, Adaptation, and Personalization (UMAP 2009)*, LNCS, Springer, vol. 5535, pp. 138-149.
- [12] Nash, SS & Rice, W 2018, *Moodle 3 E-Learning Course Development*, 4th edn, Packt Publishing, Birmingham, UK.
- [13] Nassr, RM, Saleh, AH, Dao, H & Saadat, MN 2019, 'Emotion-Aware Educational System: The Lecturers and Students Perspectives in Malaysia', in *Proceedings of IMCOM 2019, Advances in Intelligent Systems and Computing*, Springer, vol. 935, pp. 618-628.
- [14] Salmeron-Majadas, S, Santos, OC & Boticario, JG 2014, 'An evaluation of mouse and keyboard interaction indicators towards non-intrusive and low cost affective modeling in an educational context', *Procedia Computer Science*, vol. 35, pp. 691-700.
- [15] Sarrafzadeh, A, Hosseini, HG, Fan, C & Overmyer, SP 2003, 'Facial expression analysis for estimating learner's emotional state in intelligent tutoring systems', *Proceedings 3rd IEEE International Conference on Advanced Technologies*, pp. 336-337.
- [16] Tawatsuji, Y, Uno, T, Fang, S & Matsui, T 2018, 'Real-time estimation of learners' mental states from physiological information using deep learning', *Proceedings of the 26th International Conference on Computers in Education*, pp. 107-109.
- [17] Ueno, M 2005, 'Web based Computerized Testing System for Distance Education', *Educational Technology Research*, vol. 28, no. 1, pp. 59-69.
- [18] Vaswani, A, Shazeer, N, Parmar, N, Uszkoreit, J, Jones, L, Gomez, AN, Kaiser, Ł & Polosukhin, I 2017, 'Attention Is All You Need', *Advances in Neural Information Processing Systems* 30, pp. 5998-6008.
- [19] Wang, AI & Tahir, R 2020, 'The effect of using Kahoot! for learning – A literature review', *Computers & Education*, vol. 149, 103818.
- [20] Weng, C, Otanga, S, Weng, A & Cox, J 2018, 'Effects of interactivity in E-textbooks on 7th graders science learning and cognitive load', *Computers & Education*, vol. 120, pp. 172-184.